Bibliothèques 1
Streamlit 1
Json 1
Pandas 1
Altair 1
Faker 1
Numerize 1
Explication du code 2
Page app 2
Redirection 2
Chartre graphique 2
Page tableau 2
Configuration et import du fichier json 2
Organisation de la page 3
Liste d'une donnée sous forme de tableau 3
Sélection d'une donnée pour obtenir ses informations 4
Classement 5
Formatage 6
Filtrage 7
Page graphique 7
Graphique linéaire de la répartition des coûts 7
Graphique en barre 8
Graphique camembert classique 9
Graphique camembert avec angle radiant 9
Graphique camembert dynamique 9
Évolution des coûts 10
Formatage 11
Conclusion 11
Annexe 12
Documentation 12

Contexte

Le but de l'application est de pouvoir lire et afficher des données sous forme de tableaux et de graphiques pour pouvoir plus facilement évaluer les coûts ainsi que les charges de chaque élément. Pour cela on utilise python ainsi que des librairies associées. Les données ont été fourni par un export json.

Bibliothèques

Dans cet onglet nous allons évoquer toutes les bibliothèques utilisées et leur fonction dans le projet.

```
import streamlit as st
import pandas as pd
import altair as alt
from pages.Tableaux import *
```

```
import streamlit as st
import pandas as pd
import json
from faker import Faker
```

Streamlit

La bibliothèque Streamlit sert à héberger l'application sur un environnement virtuel dédié. De plus, Streamlit va gérer tout l'aspect graphique de la page avec l'affichage, ce que l'on souhaite visualiser et mettre sur la page notamment avec l'affichage des tableaux, des graphiques ou des titres.

Json

La bibliothèque Json sert à importer le fichier utilisé pour pouvoir visualiser les données.

Pandas

La bibliothèque Pandas sert à manipuler et analyser les données, c'est cette bibliothèque qui permet de faire des graphiques ou bien de pouvoir permettre à Streamlit de réaliser des graphiques ou tableaux grâce aux données traitées.

Altair

La bibliothèque Altair sert à la visualisation des données notamment sous forme de graphique. Cette bibliothèque permet d'ajouter des graphiques différents ou plus avancés que ceux de Streamlit. Elle permet plus de choix et de personnalisation.

Faker

La bibliothèque Faker sert à générer de fausses données, des données qui ne sont pas réelles. Dans le cadre de ce projet, Faker a été utilisé pour anonymiser les ressources pour répondre aux règles de la RGPD. Les ressources étant les informations personnelles des membres de l'entité telles que le nom ou le prénom.

Numerize

La bibliothèque Numerize sert à convertir des nombres avec leur suffixe, cela permet d'avoir une écriture plus compréhensible des données. Par exemple : 1000 passe à 1k. Dans le cadre de ce projet, la bibliothèque a été utiliser quelques fois notamment pour les valeurs entières.

Explication du code

Page app

Cette page est celle qui s'ouvre par défaut, elle sert à présenter le projet sous forme d'une page d'accueil. Elle possède la description de ce projet ainsi que des redirections vers les pages tableau et graphique qui servent à visualiser les données.

Redirection

Pour les redirections, j'utilise des boutons ainsi que la méthode switch_page() de Streamlit.

```python
graphique = st.button("Allez sur la page Graphique")
tableau = st.button("Allez sur la page Tableaux")
if tableau:
    st.switch_page("pages/Tableaux.py")
if graphique:
    st.switch_page("pages/Graphique.py")
```

Chartre graphique

Pour que l'application corresponde mieux à la demande, j'utilise la chartre graphique de l'entreprise pour les différents éléments à l'aide du fichier config.toml qui se situe dans le dossier à la racine .streamlit au même endroit que l'application. De plus, j'ai mis en place la possibilité de pouvoir changer entre un fond clair et un fond sombre.

.streamlit > config.toml

```toml
[theme.light]
primaryColor = "#001A496"
backgroundColor = "#f0f2f6"
secondaryBackgroundColor = "#6558b1"
textColor = "#000000"
font = "sans serif"
```

Page tableau

Cette page est celle qui permet de visualiser les données de manières brutes sous différentes formes mais principalement en tableau.

Configuration et import du fichier json

Dans un premier temps, je configure la page et je mets en place l'import du fichier json dans l'application pour pouvoir l'utiliser. Par la suite, j'utilise la bibliothèque json pour lire et exploiter les données. De plus, dans les paramètres de la page je modifie le layout pour permettre à l'application de s'adapter à la taille de la page.

```
#Ouverture du fichier json et récupération des données
with open('Woinic_suivi_projet_DEA_D2D_D2D_DEV_APPLICATIONS_TRANSPORTEURS_ANNEE-2026_21_05_2026_10_43_14.json','r', encoding='utf-8') as d :
    donnee= json.load(d)

#Configuration de la page
st.set_page_config(page_title='Tableau de données',page_icon=' ',layout="wide")
st.title('Tableau')
```

Organisation de la page

Pour rendre la navigation dans la page plus simple et pour pouvoir accéder plus facilement aux données souhaitées, j'ai mis en place un système de pills qui lorsqu'une est sélectionnée la page n'affiche que les informations liées à cette donnée avec des if. Il y a le choix entre le coût et la charge, les deux unités présentes dans le projet WOINIC. Les pills sont des éléments de sélection sous forme de boutons arrondis, permettant de représenter des catégories cliquables. Elles servent ici à filtrer les données et à ajouter des conditions afin de modifier l'affichage de la page. Les pills se situent au niveau de la sidebar pour une meilleure accessibilité.

```
pills= st.pills(
    "Choix des données affichées",
    options=["Coût","Charge"],
    selection_mode="single"
)
```

De plus, pour palier à des problèmes de la barre de scroll qui était trop longue. Pour cela j'ai mis en place un système avec des onglets (tab) qui permet d'attribuer un onglet par catégorie ce qui engendre une meilleure lecture mais aussi moins de scroll.

```
tab1, tab2, tab3, tab4, tab5, tab6 = st.tabs(["Entité", "Projet", "Phase","Activité","Profil","Ressource"])


with tab1:
```

J'ai d'abord configuré le nom des onglets souhaités puis après avec le mot clé : with nous pouvons rentrer le code que nous souhaitons afficher dans l'onglet.

J'ai également ajouté un onglet sélection dans la page graphique pour regrouper tous les filtres au même endroit.

Liste d'une donnée sous forme de tableau

Pour illustrer la méthode qui est assez similaire peu importe la donnée choisit, je vais utiliser et expliquer le code sur les profils. Tout d'abord, je nomme notre partie dans la page avec un sous-titre (st.subheader). J'initialise ensuite un tableau vide qui stockera les différents profils et leurs informations. Par la suite je fais une boucle for pour parcourir chaque couche du fichier json jusqu'à celle voulue -> suivi_profils. Ensuite j'ajoute chaque objet avec ses données dans le tableau.

```
st.subheader('Liste des profils')
liste_profils = []
for projet in donnee:
    activites_sans_phase = projet.get("suivi_activite_sans_phase")
    for activite in activites_sans_phase:
        profil_sans_phase = activite.get("suivi_profil")
        for profil in profil_sans_phase:
            liste_profils.append({
                "Responsabilité profil": profil["libelle"],
                "Nom projet": projet["nom"],
                "Nom phase": "Sans phase",
                "Nom activité": activite["libelle"],
                "Date de début": profil["date_debut"],
                "Date de fin": profil["date_fin"],
                "Charge consommée": profil["charge_consommee"],
                "Charge estimée": profil["charge_estimee"],
                "Coût consommé": profil["cout_consomme"],
                "Coût estimé": profil["cout_estime"],
                "Charge":profil["charge_estimee"]-profil["charge_consommee"],
                "Coût":profil["cout_estime"]-profil["cout_consomme"]

            })
```

Enfin, je crée le dataframe avec Pandas puis j'affiche celui-ci grâce à Streamlit. Je modifie également les paramètres pour obtenir l'affichage souhaité, avec setindex() pour modifier ce qui s'affiche en tant qu'ID dans le tableau, ou sort_values() pour trier les profils par ordre alphabétique, grâce à l'activation d'une case à cocher. J'ai également utilisé column_config pour pouvoir modifier l'affichage de certaines colonnes notamment pour avoir un formatage pour arrondir et pour ajouter l'unité.

```
tous_les_profils = pd.DataFrame(liste_profils)
```

```
alpha = st.checkbox("Ordre alphabétique par nom de profil")
if alpha:
    st.dataframe(
        tous_les_profils.set_index("Responsabilité profil").sort_values(by="Responsabilité profil"),
        column_config={
            "Charge consommée":st.column_config.NumberColumn("Charge consommée",format="%d j"),
            "Charge estimée":st.column_config.NumberColumn("Charge estimée",format="%d j"),
            "Coût consommé":st.column_config.NumberColumn("Coût consommé",format="%d €"),
            "Coût estimé":st.column_config.NumberColumn("Coût estimé",format="%d €"),
            "Charge":st.column_config.NumberColumn("Charge",format="%d j"),
            "Coût":st.column_config.NumberColumn("Coût",format="%d €")
        }
    )
```

Les colonnes Coût et Charge représentent respectivement la différence entre le coût estimé et le coût consommé, la différence entre la charge estimée et la charge

consommée. Cela permet d'avoir une notion de budget si la valeur est positive cela veut dire que nous sommes toujours dans l'estimation mais si c'est négatif cela veut donc dire que nous sommes au-dessus de l'estimation.

Sélection d'une donnée pour obtenir ses informations

J'ai mis en place une selectbox qui permet de sélectionner un élément et d'obtenir par la suite son coût, sa charge et toutes les données associées à l'élément sélectionné. Nous allons encore une fois prendre l'exemple du profil pour illustrer et expliquer le code.

```
selection= st.selectbox(
    "Quel profil voulez vous consulter ?",
    tous_les_profils["Responsabilité profil"].unique(),
    index=None,
    placeholder="Sélectionnez un profil"
)
    (variable) selection: Any | None
if selection == None:
    st.write("")
```

Dans un premier temps, je paramètre la selectbox avec son titre, la liste d'élément dans laquelle l'utilisateur doit sélectionner le profil, j'ajoute unique() pour mettre chaque profil seulement une seule fois dans la liste. Par défaut la selectbox ne sélectionne rien, si c'est le cas alors on n'affiche rien sinon on affiche du texte.

```
else:
    profil = tous_les_profils[tous_les_profils["Responsabilité profil"] == selection]
    cout_profil= profil["Coût consommé"].sum()
    cout_profil= int(cout_profil)
    charge_profil= profil["Charge consommée"].sum()
    charge_profil= int(charge_profil)
    st.write(f"Vous avez choisi : {selection}")
    st.write(f"Le coût total de {selection} est de {cout_profil} €")
    st.write(f"La charge totale de {selection} est de {charge_profil} j")
    st.write("")
    ressources = toutes_les_ressources[toutes_les_ressources["Responsabilité profil"] == selection]
    noms = ressources["Nom complet"].unique()
    st.write("Les ressources associées sont les suivantes :")
    for nom in noms:
        st.write("-" + nom)
    st.dataframe(pd.DataFrame(profil))
```

Si un nom de profil présent dans la liste profil est égal à la sélection alors j'ajoute aux différentes variables la somme des coûts associés. Je parcours également les autres données qui peuvent être associées au profil pour ensuite les écrire. Pour finir, je fais un dataframe pour pouvoir afficher tous les profils en détail de la liste qui possède le

profil sélectionné par l'utilisateur. Si je suis sur une autre couche du fichier json, il suffit juste d'adapter le code pour parcourir et afficher jusqu'à la couche voulut.

Classement

Je vais maintenant présenter la dernière fonctionnalité de la page tableau qui est celle du classement. Ce code permet de classer les profils par ordre décroissant de coût.

```
#Classement profil les plus couteux (ordre décroissant) avec profil regroupé
st.subheader("Classement profil (€)")
resultat_profil= {}
for profil in liste_profils:
    nom = profil["Responsabilité profil"]
    cout = profil["Coût consommé"]

    if nom in resultat_profil:
        resultat_profil[nom] += cout
    else:
        resultat_profil[nom] = cout
classement_profil_cout = pd.DataFrame(list(resultat_profil.items()), columns=["Responsabilité profil", "Coût consomm
st.dataframe(classement_profil_cout.set_index("Responsabilité profil").sort_values(by="Coût consommé", ascending=Fa
             column_config={
                 "Responsabilité profil": st.column_config.Column("Responsabilité profil"),
                 "Coût consommé":st.column_config.NumberColumn("Coût consommé",format="%d €")
             })
```

Tout d'abord j'initialise un dictionnaire qui nous permettra d'associer une clé nom à sa valeur cout. J'utilise une boucle for pour parcourir la liste de profil, pour chaque profil je stocke momentanément dans une variable son nom et son coût, si le nom est déjà dans le tableau alors j'ajoute son coût au précédent sinon je le rentre dans le dictionnaire. Ensuite je crée le dataframe en paramétrant les colonnes de celui-ci et récupérant les informations que je peux traiter à l'aide de items(). Enfin, j'affiche le tableau à l'aide de Streamlit puis je tri celui-ci par ordre décroissant pour en faire un classement à l'aide de sort_values().

Formatage

Pour que les données chiffrées apparaissent avec leurs suffixes j'ai dû effectuer un formatage des données notamment en séparant les tableaux d'affichage et ceux qui permettent de concevoir les graphiques car les graphiques ont besoin de données chiffrées. Pour cela, j'ai dans un premier temps réalisé une fonction qui prend en paramètre le Dataframe dont nous voulons les valeurs exprimées avec des suffixes à la fin ainsi qu'un tableau des éléments auquel nous voulons appliquer la fonction.

```
#Fonction de conversion en k
def convertir_en_k(dataframe,colonnes):
    dataframe= dataframe.copy()
    for col in colonnes:
        dataframe[col] = dataframe[col] /1000
    return dataframe
```

Dans un premier temps, je copie le Dataframe pour ne pas modifier celui qui est nécessaire pour les graphiques. Ensuite, je parcours celui-ci avec les colonnes en paramètre puis je remplace le contenu de chaque colonne par sa valeur divisée par 1000. Je finis par renvoyer le Dataframe pour que celui-ci soit récupérer par une

variable et donc par la suite utiliser. Enfin, pour appliquer le formatage nous allons faire des columns config pour pouvoir sélectionner celle que je souhaite afficher ainsi que les valeurs. Dans le cas suivant, j'utilise deux types de format, le premier étant %d qui sert à tronquer la valeur ainsi qu'ajouter l'unité à la fin, le deuxième étant %.3f qui permet d'avoir un nombre réel avec 3 décimales. Le formatage est appliqué pour chaque donnée mais varie en fonction de l'unité.

```
st.dataframe(
    liste_entites_affichage.set_index("Entite").sort_values(by="Entite"),
    column_config={
        "Charge consommée":st.column_config.NumberColumn("Charge consommée",format="%d j"),
        "Charge estimée":st.column_config.NumberColumn("Charge estimée",format="%d j"),
        "Coût consommé":st.column_config.NumberColumn("Coût consommé",format="%.3f k €"),
        "Coût estimé":st.column_config.NumberColumn("Coût estimé",format="%.3f k €"),
        "Charge":st.column_config.NumberColumn("Charge",format="%d j"),
        "Coût":st.column_config.NumberColumn("Coût",format="%.3f k €")
    }
)
```

Filtrage

J'ai également implémenté une notion de filtrage qui est le filtre par ordre alphabétique pour chacune des données respectives. Les filtres sont regroupés dans la sidebar. J'utilise une case à cocher pour le filtrage, si celle-ci est actionnée alors le tableau est trié par ordre alphabétique sinon il reste par rapport à l'ordre du fichier. Pour trier par ordre alphabétique, j'utilise sort_values() avec entre parenthèse le nom de l'élément que je souhaite trier. Pour vérifier que cela fonctionne je mets une condition avant l'affichage du graphique.

```
with st.sidebar:
    st.subheader("Filtre")
    alpha = st.checkbox("Ordre alphabétique par nom d'entité")
#Liste entité
st.subheader('Liste des entités')
if alpha:
    st.dataframe(
        liste_entites_affichage.set_index("Entite").sort_values(by="Entite"),
```

Page graphique

Cette page sert à exploiter les données pour les mettre sous forme de graphique pour avoir une meilleure visualisation. La page reprend également la même organisation que celle avec les tableaux.

Graphique linéaire de la répartition des coûts

Je récupère le dataframe du tableau pour pouvoir en faire un graphique linéaire (mark_line()), j'exploite les données que j'ai précédemment récupérées à l'aide des tableaux. Tout d'abord, je mets en paramètre le dataframe ensuite je mets les valeurs de

x et de y que je souhaite affichée ici le nom de l'entité et son coût. J'utilise également astype(int) pour tronquer la valeur et avoir un entier.

```
#Graphique coût entité
st.subheader("Répartition des coûts des différentes entités (€)")
graphe_coût_entite= pd.DataFrame(liste_entites)
graphe_coût_entite ["Coût consommé"] = graphe_coût_entite ["Coût consommé"].astype(int)
chart_coût_entite= alt.Chart(graphe_coût_entite).mark_line().encode(
  x="Entite",
  y=alt.Y("Coût consommé", title="Coût consommé (en €)",axis=alt.Axis(format="~s"))
)
st.altair_chart(chart_coût_entite)
```

Graphique en barre

J'utilise la bibliothèque altair pour pouvoir réaliser le graphique car elle permet de pouvoir avoir plus de paramètre et plus de liberté qu'un bar_chart classique sur Streamlit. Ce graphique montre le coût des différentes phases par rapport à son projet. J'utilise sort() pour trier les projets par ordre décroissant de leur coût. Ensuite, je choisis les différentes couleurs du graphique qui représentent les différentes phases pour chaque projet avec alt.Color(). Enfin, j'utilise alt.Legend() pour pouvoir modifier la légende et la rendre plus visible et homogène.

```
    chart_coût_phase = alt.Chart(filtre).mark_bar().encode(
    x=alt.X("Coût consommé", title="Coût consommé (en €)"),
    y=alt.Y("Nom projet",sort="-x"),
    color=alt.Color(
        "Nom phase",
        legend=alt.Legend(
            title="Nom phase",
            orient="bottom",
            columns=3,
            symbolLimit=60
        )
    )
)

st.altair_chart(chart_coût_phase)
```

Les graphiques ont une spécificité qui est que les données que vous voulez afficher peuvent être sélectionner dans une multiselectbox, si la liste de sélection n'est pas vide alors le code renvoi uniquement le graphique avec les données sélectionnées

sinon il renvoi le graphique avec l'ensemble des données.

```
st.subheader('Répartition des coûts des différentes phases (€)')
graphe_coût_phase = pd.DataFrame(tab_phase)
graphe_coût_phase ["Coût consommé"] = graphe_coût_phase ["Coût consommé"].astype(int)
choix_phases_cout = st.multiselect(
    "Choisir les phases",
    liste_phase["Nom phase"].unique(),
    placeholder="Sélectionnez une phase ou plusieurs phases")

filtre = graphe_coût_phase[(graphe_coût_phase["Nom phase"].isin(choix_phases_cout)) & (graphe_coût_phase["Coût c
filtre_0 = graphe_coût_phase[graphe_coût_phase["Coût consommé"] > 0]
```

Je paramètre la multiselect, pour cela je lui choisis un titre, je rentre les éléments que je souhaite avoir donc ici le nom des phases unique. Ensuite, j'ai mis en place des filtres pour gérer les deux situations possibles : L'utilisateur a sélectionné au moins un élément dans la multiselectbox et au contraire, l'utilisateur n'a pas saisi un élément. Le filtre_0 est le filtre quand il n'y a pas de sélection et qu'on renvoie donc tout par défaut à condition que le coût soit supérieur à 0. Au contraire filtre permet en plus de vérifier si ce qui a été sélectionner existe et si c'est le cas alors cela renvoie les données dans le graphique.

Graphique camembert classique

Pour faire le graphique camembert j'utilise une fois de plus la bibliothèque altair qui offre plus d'options et de choix. Tout d'abord, je trie les valeurs pour reproduire le classement ensuite je mets les valeurs en entier puis pour finir je sélectionne uniquement les 10 premiers éléments à l'aide de head().

```
#Camembert phase coût
st.subheader("Les 10 phases avec le coût le plus élevé (€)")
classement_phase_cout = classement_phase_cout.sort_values(by="Coût consommé",ascending=False)
classement_phase_cout ["Coût consommé"] = classement_phase_cout ["Coût consommé"].astype(int)
top_10_phase_cout = classement_phase_cout.head(10)
pie_phase_cout = alt.Chart(top_10_phase_cout).mark_arc().encode(
    theta=alt.Theta(field="Coût consommé", type="quantitative"),
    color=alt.Color(field="Nom phase", type="nominal",legend=alt.Legend(
        title="Phase",
        orient="bottom",
        columns=2,
        symbolLimit=30
    )),
).properties(height=700)
st.altair_chart(pie_phase_cout)
```

Graphique camembert avec angle radiant

```
st.subheader("Les 10 projets avec le coût le plus élevé (€)")
classement_projet_cout = classement_projet_cout.sort_values(by="Coût consommé",ascending=False)
classement_projet_cout ["Coût consommé"] = classement_projet_cout ["Coût consommé"].astype(int)
top_10_projet_cout = classement_projet_cout.head(10)
pie_projet_cout = alt.Chart(top_10_projet_cout).mark_arc(innerRadius=30, stroke="#fff").encode(
    theta=alt.Theta(field="Coût consommé", type="quantitative", stack=True),
    radius=alt.Radius("Coût consommé", scale=alt.Scale(type="sqrt", zero=True, rangeMin=20)),
    color=alt.Color(field="Nom projet",legend=alt.Legend(
        title="Projet",
        orient="bottom",
        columns=2,
        symbolLimit=30
    )),
).properties(height=700)
st.altair_chart(pie_projet_cout)
```

C'est la même chose que le graphique précédent mais j'ajoute quelques paramètres comme au niveau du mark_arc() pour ajouter une surbrillance ou alors l'ajout du paramètre radius() qui permet d'avoir un graphique à angle radiant.

Graphique camembert dynamique

Ce graphique a pour but de faire choisir le nombre de données que l'utilisateur veut afficher, il reprend exactement la même logique que les graphiques précédents mais avant je demande une saisie pour pouvoir rendre dynamique certains éléments.

```
#Camembert entité coût
st.subheader("Saisie pour déterminier le graphique de répartition ci-dessous")
dynamique = st.text_input("Saisissez un chiffre entre 1 et 9 pour établir le nombre d'entité que vous voulez sur
dynamique=int(dynamique)
if dynamique ==1:
    st.subheader(f"L'entité avec le coût le plus élevé (€)")
elif dynamique > 1 and dynamique <=9:
    st.subheader(f"Les {dynamique} entités avec le coût le plus élevé (€)")
if dynamique < 1 or dynamique > 9:
    st.write("Erreur, vous devez saisir un entier compris entre 1 et 9")
else:
    classement_entite_cout ["Coût consommé"] = classement_entite_cout ["Coût consommé"].astype(int)
    top_10_entite_cout = classement_entite_cout.head(dynamique)
    pie_entite_cout = alt.Chart(top_10_entite_cout).mark_arc().encode(
        theta=alt.Theta(field="Coût consommé", type="quantitative"),
        color=alt.Color(field="Entite", type="nominal",legend=alt.Legend(
            title="Entité",
            orient="bottom",
            columns=2,
            symbolLimit=30
        )),
).properties(height=700)
```

On peut voir que le titre ainsi que le classement sont dynamiques à l'aide de la variable dynamique saisie par un text_input(). On peut voir la variable dans le head() et dans le titre. De plus, j'ai mis des conditions car il existe un maximum de 9 entités et un minimum de 1 entité si ce n'est pas le cas de la saisie alors cela renverra une erreur.

Évolution des coûts

Par la suite, j'ai réalisé un graphique qui représente les coûts pour chaque mois et un second pour montrer l'évolution des coûts au fur et à mesure des mois. Dans un premier temps, j'ai fait une selectbox pour sélectionner le projet dont on souhaite consulter l'évolution.

```python
st.subheader("Evolution des coûts d'un projet (€)")
selection= st.selectbox(
    "Quel projet voulez vous voir l'évolution ? (coût)",
    liste_projet_conso_mois["Nom projet"].unique(),
    index=None,
    placeholder="Sélectionnez un projet"
)
```

```python
#Graphique consommation chaque mois
projet_conso = liste_projet_conso_mois[liste_projet_conso_mois["Nom projet"] == selection].iloc[0]
st.subheader("Répartition des coûts par mois du projet selectionné")
graphe_coût_projet_mois= pd.DataFrame(projet_conso)
dates = pd.date_range(start="2026-01-01", periods=12, freq="MS")
cout = [projet_conso[i] for i in mois]
cout_par_mois = pd.DataFrame({"Mois": dates, "Cout par mois (€)": cout})
cout_par_mois ["Cout par mois (€)"] = cout_par_mois ["Cout par mois (€)"].astype(int)
st.line_chart(cout_par_mois.set_index("Mois"))

#Graphique évolution de la consommation par mois
st.subheader("Évolution du coût par mois du projet selectionné")
dates = pd.date_range(start="2026-01-01", periods=12, freq="MS")
cout = [projet_conso[i] for i in mois]
cout_par_mois = pd.DataFrame({"Mois": dates, "Cout par mois": cout})
cout_par_mois["Cumul (€)"]= cout_par_mois["Cout par mois"].cumsum()
cout_par_mois ["Cumul (€)"] = cout_par_mois ["Cumul (€)"].astype(int)
st.line_chart(cout_par_mois.set_index("Mois")["Cumul (€)"])
```

Les deux programmes sont quasiment identiques, mais l'un possède un champ supplémentaire permettant de stocker la variable de cumul au fur et à mesure de l'avancement. La variable dates correspond à l'axe x. À l'aide de la méthode date_range, on peut définir une date de début — ici le 1er janvier — et lui attribuer une fréquence (MS pour début de mois) ainsi qu'un nombre de périodes afin de déterminer la durée et la répétition, ce qui permet de visualiser l'évolution.

Ensuite, la variable cout permet d'extraire les informations du projet sélectionné, stockées dans projet_conso, correspondant à la sélection. La variable mois est un tableau contenant tous les mois ; si un élément est présent, il est alors stocké dans cout sous forme de tableau.

Par la suite, on crée un DataFrame avec les colonnes dates et cout. Enfin, pour le second programme, la colonne cumul correspond à la somme cumulative des valeurs de cout par mois : celles-ci s'additionnent au fur et à mesure. Pour cela, on utilise la méthode cumsum()

Formatage

Je vais également formater les données des graphiques pour que cela convertisse pour cela j'utilise toujours format mais avec une valeur différente : format="~s", cela permet la conversion automatique en K, M mais cela ne peut être utilisé uniquement pour les

graphiques. J'utilise également un tooltip cela permet d'afficher les informations souhaitées quand on passe sur une partie du graphique.

```
tooltip= [alt.Tooltip("Coût consommé",title="Coût consommé (€)",format="~s"),
         alt.Tooltip("Entite",title="Entite")]
```

Conclusion

Ce projet avait pour but de développer un outil pour pouvoir visualiser les coûts et les charges des différents éléments. Les principales fonctionnalités de l'application étant la visualisation par graphique ou par tableau des données à l'aide de Streamlit. J'ai mis en place des listes, des classements, différents types de graphique, des listes de sélection mais aussi d'autres fonctionnalités mineures pour une meilleure compréhension de l'outil.

Annexe

Documentation

Cet onglet sert à présenter les documentations et sources majeures que j'ai utilisé durant le projet.

Dans un premier temps j'ai utilisé le site et la documentation officielle de Strealit : https://docs.streamlit.io/develop/api-reference

J'ai également utilisé une application Streamlit qui présente tous les types de graphiques altair possibles avec un exemple de graphique et son code : https://altair.streamlit.app/

Pour finir, je me suis également servit des documentations des différentes bibliothèques utilisées :

Faker : https://pypi.org/project/Faker/

Pandas : https://pandas.pydata.org/docs/